
LAB_06 : Shell Script

1. Write a shell script to display 'this is CET479' output on the command line.

`echo "This is CET479"` : display a message

```
(van@kali)-[~]  
$ echo "This is CET479"  
This is CET479
```

`$ [ chmod +x cet479.sh ]` : Save the file with a .sh extension

`$ [ ./cet479.sh ]` : Run the script of the message "this is CET479"

2. Now to the above script append 2 # signs. Check the output. Did it change?

To append use the following

`cat >> scriptname`

`#`

`#`

Ctrl +d to exit.

`#!/bin/bash`

`echo "This is CET479"` : display a message

`#`

`#`



```
(van@kali)-[~]  
$ cat>>myscript  
  
#!/bin/bash  
  
echo " This is CET479 "  
  
#  
#  
  
(van@kali)-[~]  
$
```

```
$[cat>>cet479.sh]
```

```
#  
#
```

The # character at the beginning of each line indicates that the line is a comment and will be ignored by the script interpreter.

3. Create a variable NAME=yourname. Use the same script to display the output. Again, you can use the append function. Show that your script works.

\$ [varname="Van"] : To declare a variable named "varname" with the value "Van"

```
(van@kali)-[~]  
$ varname="van"  
  
(van@kali)-[~]  
$
```



\$ [**echo** \$varname] : To display the value of a shell variable.

```
(van@kali)-[~]  
$ echo $varname  
van  
  
(van@kali)-[~]  
$
```

\$ [**varname+=** "This is used the append function"] : add a string to a the end of variable "varname"

```
(van@kali)-[~]  
$ varname+=" This is used the append funtion"  
  
(van@kali)-[~]  
$ echo $varname  
van This is used the append funtion
```

4. Use the vim editor to practice creating a script to display output of a variable (any variable of your choice) and then run your script. Show the output here.

\$ [**vim myscript**] : Creating a file vim editor => open page of file

- Use key " **i** " for enter text (insert).
#!/bin/bash
echo " Hello everyone class CET 479 Spring 2023 "
- Use key " **esc** " exit the mode insert .
- Use " **:wq** " save and quit the page enter data.



\$ [**chmod +x myscript**] : To be able to run the script without specifying the interpreter from the command line you'll need to make the file executable

\$ [**./myscript**] : run the file myscript

```
[vannguyen@fedora ~]$ vim myscript
[vannguyen@fedora ~]$ chmod +x myscript
[vannguyen@fedora ~]$ ./myscript
Hello everyone class CET 479 spring 2023
[vannguyen@fedora ~]$
```

Note: For the following questions 5, 6 and 7, you do not have to write a script. You can test in your command line interface directly.

5. Create a temporary directory and a file within it. Type ls to see that the file exists. Now type the following:

```
[vannguyen@fedora ~]$ mkdir TEMPORARY
[vannguyen@fedora ~]$ cd TEMPORARY
[vannguyen@fedora TEMPORARY]$ touch question_5
[vannguyen@fedora TEMPORARY]$ ls
question_5
[vannguyen@fedora TEMPORARY]$
```

\$ **filelist=\$(ls)** : Assigns the output of the ls command to the variable filelist

\$ **echo \$filelist** : Prints the value of the variable filelist to the terminal. The \$ sign before the variable name is used to reference its value.

```
[vannguyen@fedora TEMPORARY]$ filelist=$(ls)
[vannguyen@fedora TEMPORARY]$ echo $filelist
question_5
[vannguyen@fedora TEMPORARY]$
```



What do you observe?

the output is name of file stored in directory currents the variable .

6. Test the following expressions and write the outputs that you observe:

```
$ value=expr 1 + 2
```

```
$ echo $value
```

```
(van@kali)-[~]  
$ value=expr 1 + 2  
3  
  
(van@kali)-[~]  
$ echo $value  
expr 1 + 2
```

```
$ value = "expr 1 + 2"
```

```
$ echo $value
```

```
(van@kali)-[~]  
$ value="expr 1 + 2"  
  
(van@kali)-[~]  
$ echo $value  
expr 1 + 2
```

```
$ value = `expr 1 + 2`
```

```
$ echo $value
```



```
(van@kali)-[~]  
$ value='expr 1 + 2'  
  
(van@kali)-[~]  
$ echo $value  
expr 1 + 2
```

```
$ value = ` expr 1 + 2 `  
$ echo $value
```

```
(van@kali)-[~]  
$ value=`expr 1 + 2`  
  
(van@kali)-[~]  
$ echo $value  
3
```



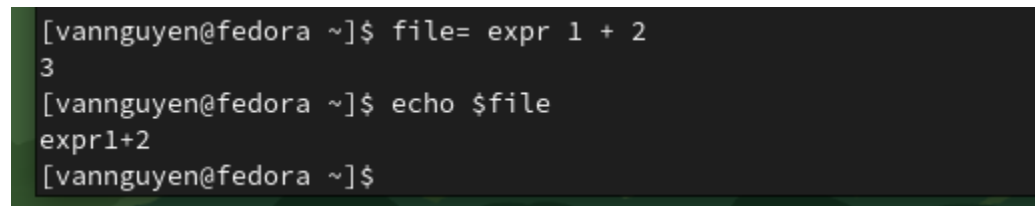
7. Now in your terminal type the following commands:

```
file=1+2
```

```
echo $file
```

```
$ file=expr 1 + 2
```

```
$ echo $file
```



```
[vannguyen@fedora ~]$ file= expr 1 + 2
3
[vannguyen@fedora ~]$ echo $file
expr1+2
[vannguyen@fedora ~]$
```

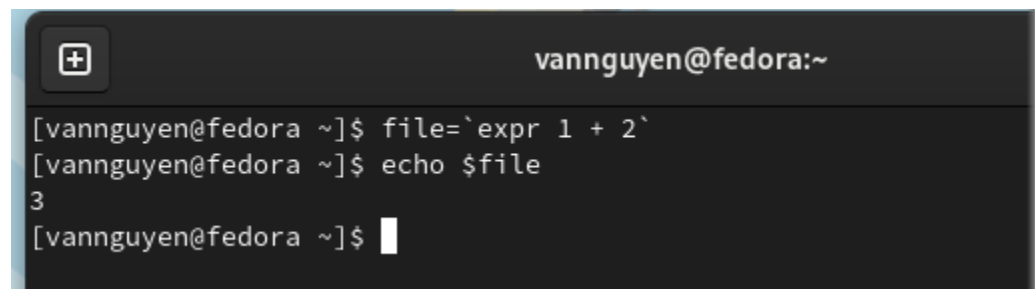
What did you observe?

Result of expression 1 + 2

Can you fix the commands to display sum of 1 & 2.

```
$ file=`expr 1 + 2`
```

```
$ echo $file
```



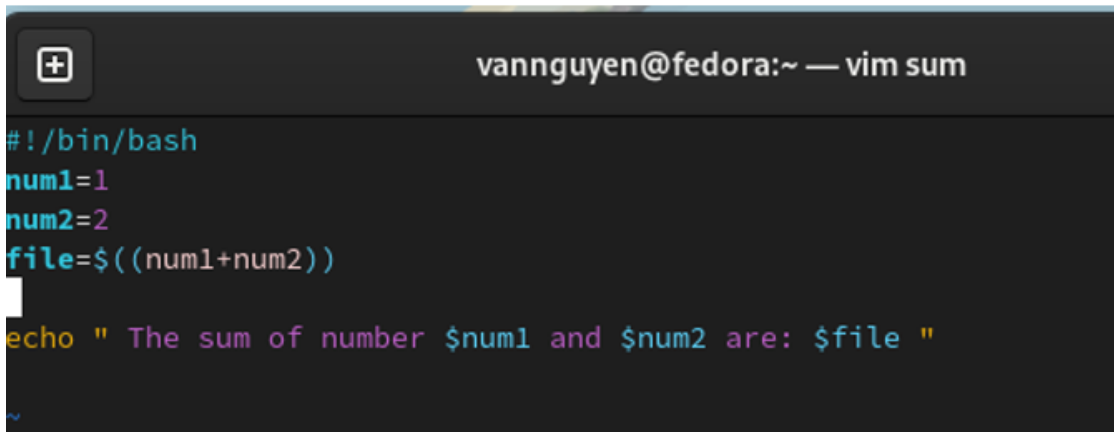
```
vannguyen@fedora:~
[vannguyen@fedora ~]$ file=`expr 1 + 2`
[vannguyen@fedora ~]$ echo $file
3
[vannguyen@fedora ~]$
```



8. Using the vim editor, create a script to print the sum of two numbers on the screen. The output should be properly displayed - using sentences like "the numbers are ... and ...". and "the sum is....."

\$ [**vim** **sum**] : Creating a file vim editor => open page of file type :

```
#!/bin/bash
num1=1
num2=2
file=$((num1+num2))
echo "The sum of number $num1 and num2 are : $file"
```



\$ [**chmod** **+x** **sum**]

\$ [**./** **sum**] : run the file sum

